

## COMMONWEALTH OF AUSTRALIA

### *Copyright Regulations 1969*

#### WARNING

This material has been reproduced and communicated to you by or on behalf of Monash University pursuant to Part VB of the *Copyright Act 1968* (the Act). The material in this communication may be subject to copyright under the Act. Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act. Do not remove this notice.



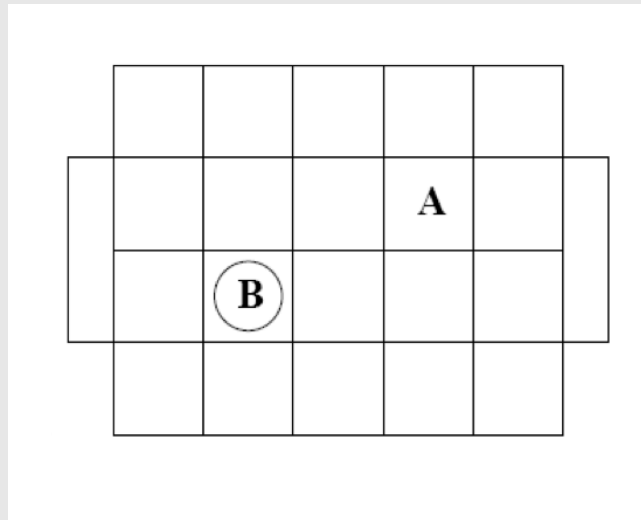


FIT5226

Multi-agent System & Collective Behaviour

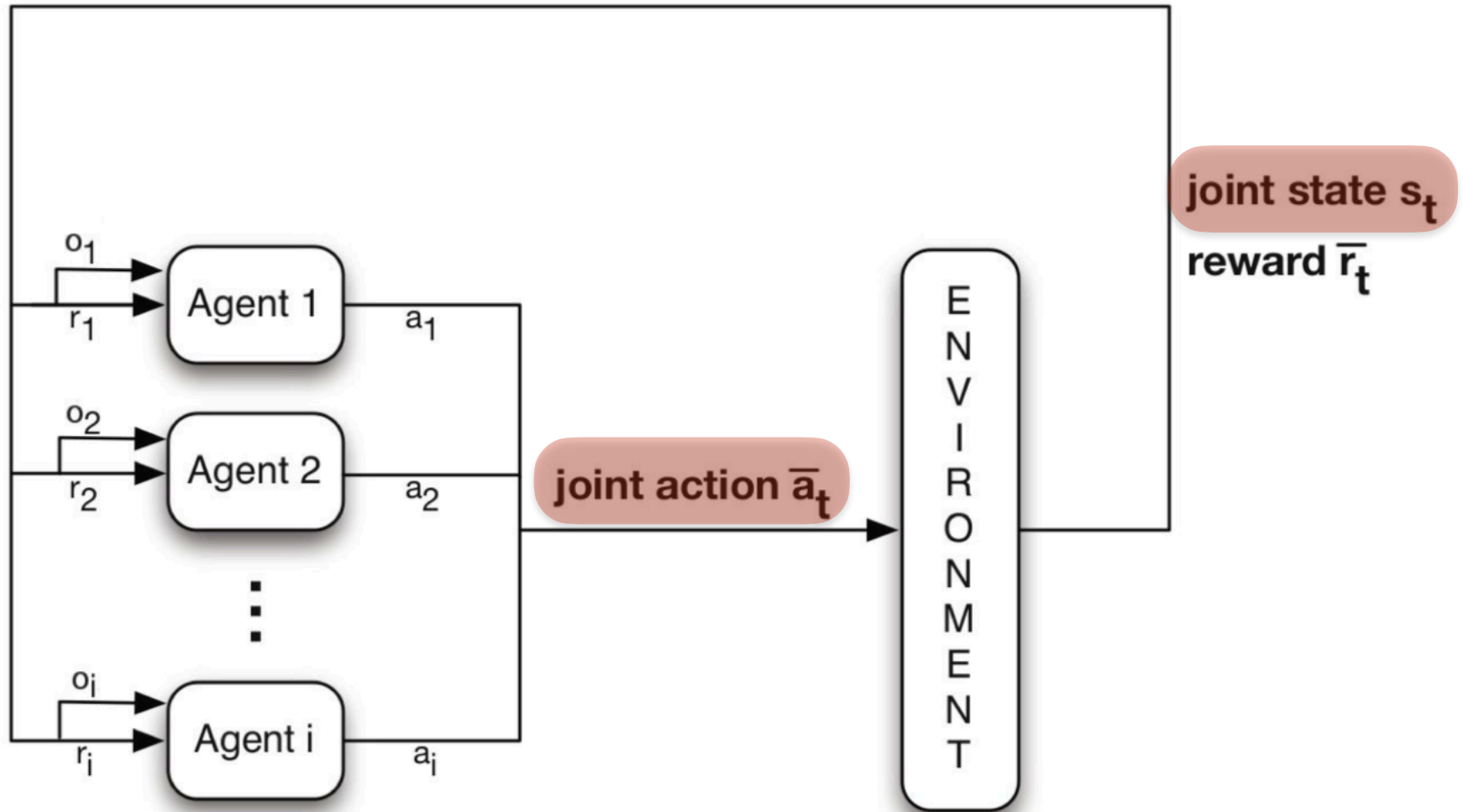
Wk 7: MiniMax Q-Learning

# Robo Soccer



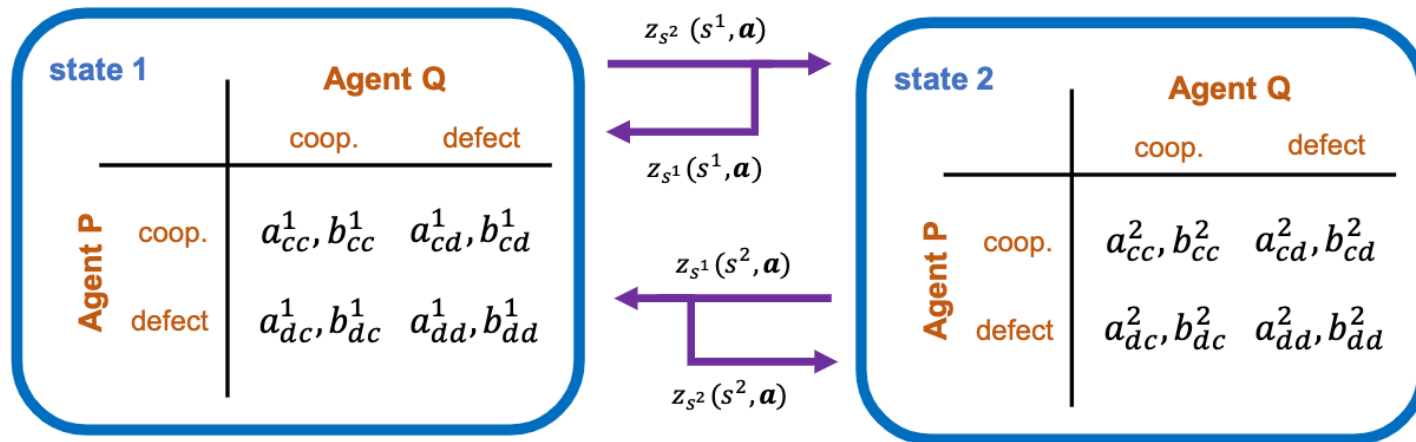
Here, we have an environment *state*  
and the other agent influences the state

# Multi-Agent-Environment Interaction



Source: Nowe, Vrancx & De Hauwere 2012

# Markov Games (*every state is a different game*)



**Figure 1: The framework of a two-state two-player two-action stochastic game. There are two players,  $P$  and  $Q$ . In every round, they can be in one state,  $s^1$  or  $s^2$ . In each state, each player has a corresponding strategy and payoff matrix. The state in the next round depends on the current state, on the players' joint action and on chance,  $z_{s'}(s, a)$ .**

# Markov Game: Definition

**Definition 6.2.1 (Stochastic game)** A stochastic game (*also known as a Markov game*) is a tuple  $(Q, N, A, P, r)$ , where:

- $Q$  is a finite set of games;
- $N$  is a finite set of  $n$  players;
- $A = A_1 \times \cdots \times A_n$ , where  $A_i$  is a finite set of actions available to player  $i$ ;
- $P : Q \times A \times Q \mapsto [0, 1]$  is the transition probability function;  $P(q, a, \hat{q})$  is the probability of transitioning from state  $q$  to state  $\hat{q}$  after action profile  $a$ ; and
- $R = r_1, \dots, r_n$ , where  $r_i : Q \times A \mapsto \mathbb{R}$  is a real-valued payoff function for player  $i$ .

# Nash in Markov Games

---

Theorem:

Every two-player, constant-sum, average reward, irreducible Markov game has a Nash equilibrium

*“irreducible” means here that the Markov chain of games must be irreducible, in other words, every game must always remain reachable, regardless of which strategies are played.*

# Standard Q-Learning

Recall standard Q-Learning (here in the formulation from SLB).

This can be adapted to 2-player zero-sum games relatively immediately.

**Definition 7.4.1 (*Q-learning*)** *Q-learning is the following procedure:*

Initialize the  $Q$ -function and  $V$  values (arbitrarily, for example)

**repeat** *until convergence*

    Observe the current state  $s_t$ .

    Select action  $a_t$  and take it.

    Observe the reward  $r(s_t, a_t)$

    Perform the following updates (and do not update any other  $Q$ -values):

$$Q_{t+1}(s_t, a_t) \leftarrow (1 - \alpha)Q_t(s_t, a_t) + \alpha(r(s_t, a_t) + \beta V_t(s_{t+1}))$$

$$V_{t+1}(s) \leftarrow \max_a Q_t(s, a)$$



# Q-value derivation for known *joint* action

$$Q_{i,t+1}(s_t, a_t, o_t) = (1 - \alpha_t)Q_{i,t}(s_t, a_t, o_t) + \alpha_t(r_i(s_t, a_t, o_t) + \beta V_t(s_{t+1}))$$

$a_t$  is the player's own action,  $o_t$  that of the opponent.

Nothing has changed except for switching from a single agent's action to the joint action.

This means that the player needs to know the other player's action to perform the update.

# State value update according to MiniMax

But we don't know the actions of the other player!

The *core of the method* is to calculate the value according to the standard MiniMax method for 2-player zero-sum games.

$$V_t(s) = \max_{\Pi_i} \min_o Q_{i,t}(s, \Pi_i(s), o).$$

In practice, this requires the use of a linear solver within the Q-iteration.

# The MiniMax-Q algorithm

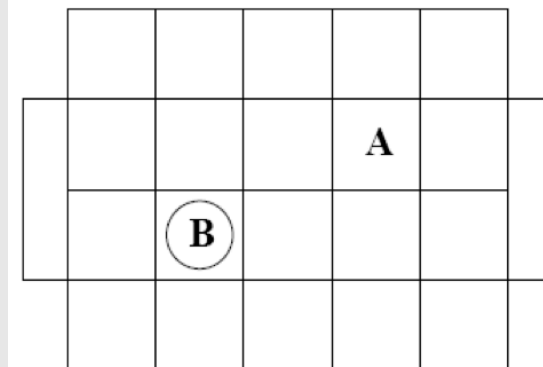
```
// Initialize:
forall  $s \in S, a \in A$ , and  $o \in O$  do
   $Q(s, a, o) \leftarrow 1$ 
forall  $s$  in  $S$  do
   $V(s) \leftarrow 1$ 
forall  $s \in S$  and  $a \in A$  do
   $\Pi(s, a) \leftarrow 1/|A|$ 
 $\alpha \leftarrow 1.0$ 

// Take an action:
when in state  $s$ , with probability explor choose an action uniformly at random,
and with probability  $(1 - \text{explor})$  choose action  $a$  with probability  $\Pi(s, a)$ 

// Learn:
after receiving reward rew for moving from state  $s$  to  $s'$  via action  $a$  and
opponent's action  $o$ 
 $Q(s, a, o) \leftarrow (1 - \alpha) * Q(s, a, o) + \alpha * (\text{rew} + \gamma * V(s'))$ 
 $\Pi(s, \cdot) \leftarrow \arg \max_{\Pi'(s, \cdot)} (\min_{o'} \sum_{a'} (\Pi(s, a') * Q(s, a', o')))$ 
// The above can be done, for example, by linear programming
 $V(s) \leftarrow \min_{o'} (\sum_{a'} (\Pi(s, a') * Q(s, a', o')))$ 
Update  $\alpha$ 
```

Figure 7.8: The minimax- $Q$  algorithm.

## Example: Grid Soccer Defense

[illegible]

# Convergence and Limitations

**Theorem 7.4.3** *Under the same conditions that assure convergence of  $Q$ -learning to the optimal policy in MDPs, in zero-sum games Minimax- $Q$  converges to the value of the game in self play.*

- **MiniMax- $Q$**  converges but only in 2-player zero-sum Games
- **Friend-or-Foe- $Q$**  extends this to general sum  $n$ -player games, but the other players must be known to the agent as either “friend” (playing for coordinated equilibrium) or “foe” (playing for adversarial equilibrium), so in a sense this is a 2-player game again, just in teams.
- More general algorithms exist but they are only guaranteed to converge only under very strict conditions
- No known solution for general-sum,  $n$ -player games